Daily PM2.5 Time Series with Gaussian Processes

This analysis uses a dataset from the daily concentration of of PM2.5 in the air at one particular monitoring station. The data is at the dataset is located at `data/station_complete.csv`. The response is transformed to the logarithm scale since it stabilizes the positive, right-skewed concentration scale.

For this analysis you will need the following R packages

Exploratory data analysis

```
d <- read.csv("data/station_complete.csv") |> mutate(date=as.Date(date)) |> arrange(day)

y <- d$log_pm25
n <- length(y)
t <- d$day
summary(d[c("pm25","log_pm25")])
```

```
      pm25          log_pm25
 Min.   :11.14   Min.   :2.410
 1st Qu.:18.71   1st Qu.:2.929
 Median :21.55   Median :3.070
 Mean   :23.17   Mean   :3.105
 3rd Qu.:26.62   3rd Qu.:3.282
 Max.   :49.61   Max.   :3.904
```

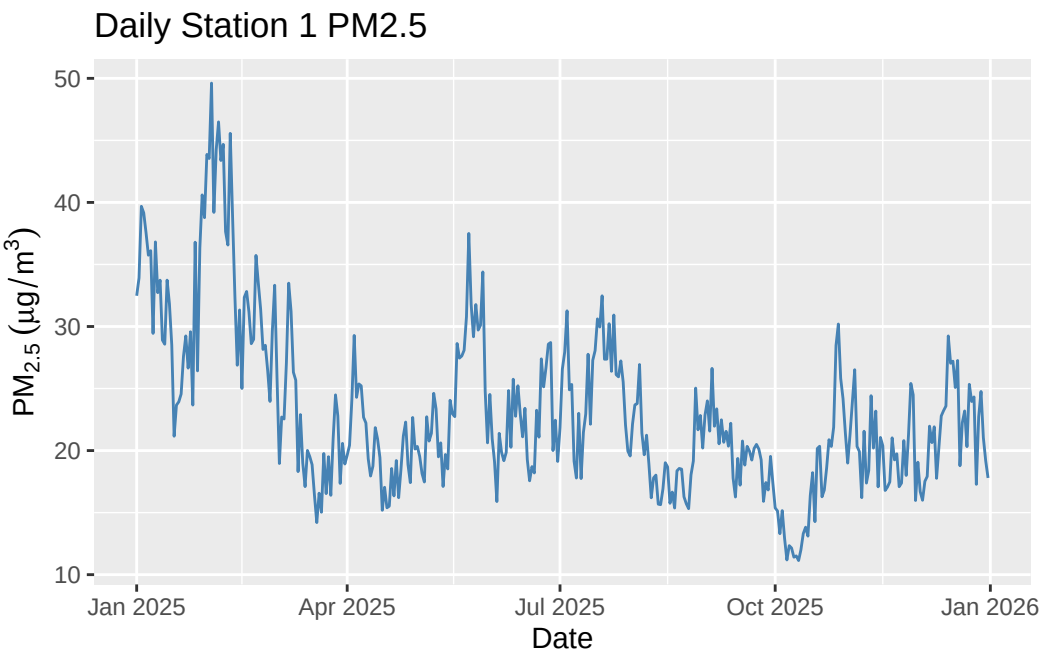
```
sum(is.na(y))
```

```
[1] 0
```

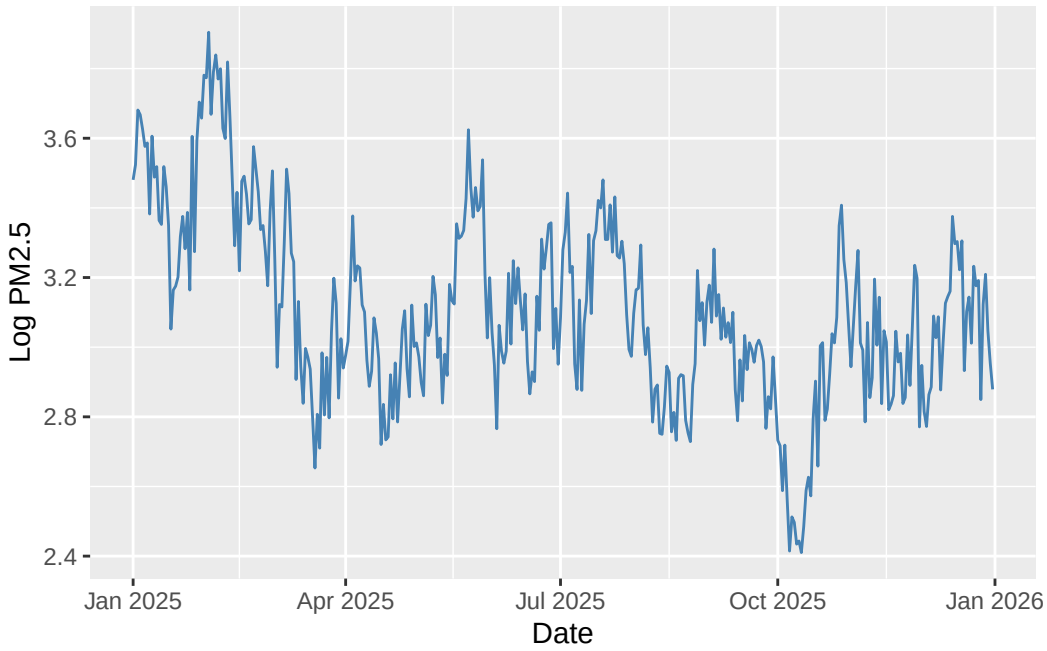
EDA

Figure of the time series

```
ggplot(d,aes(date,pm25)) + geom_line(color="steelblue") + labs(x="Date", y=expression(PM[2.5]
title="Daily Station 1 PM2.5")
```



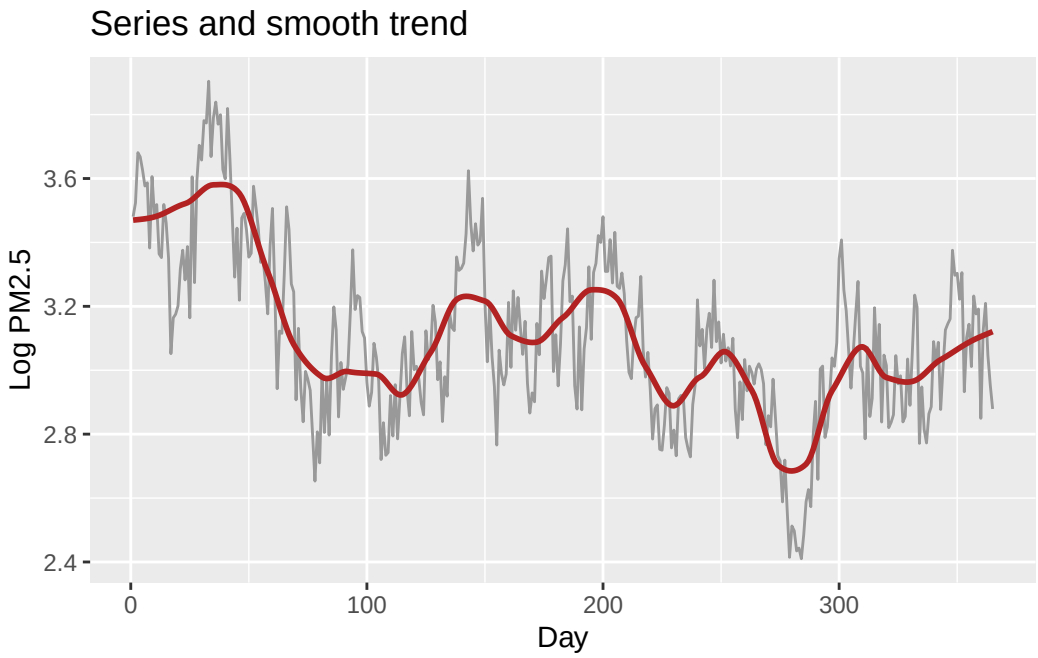

```
ggplot(d,aes(date,log_pm25))+geom_line(color="steelblue")+labs(x="Date",y="Log PM2.5")
```



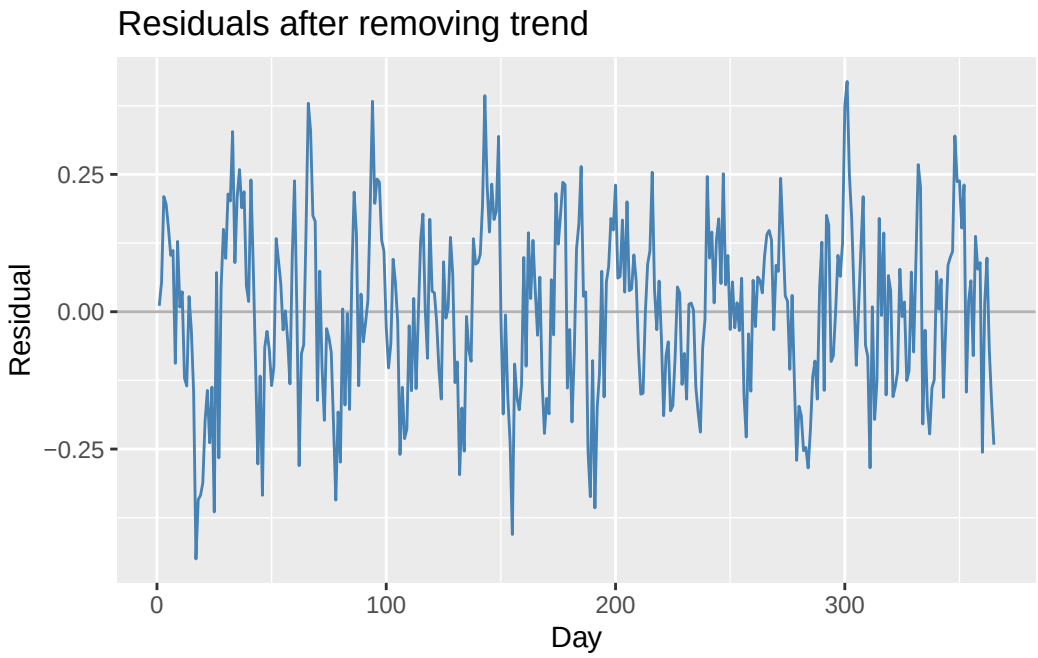
Let's explore the components of the time series: systematic + residual component. We can estimate the trend using a loess smoother. The residuals are the difference between the observed log PM2.5 and the estimated trend.

```
d$trend <- predict(
  loess(log_pm25 ~ day, data = d, span = 0.2)
)
d$residual <- d$log_pm25 - d$trend

ggplot(d, aes(day, log_pm25)) +
  geom_line(color = "grey60") +
  geom_line(aes(y = trend), color = "firebrick", linewidth = 1) +
  labs(x = "Day", y = "Log PM2.5", title = "Series and smooth trend")
```




```
ggplot(d, aes(day, residual)) +
  geom_hline(yintercept = 0, color = "grey70") +
  geom_line(color = "steelblue") +
  labs(x = "Day", y = "Residual", title = "Residuals after removing trend")
```



The functions 'decompose' and 'stl' are also available for decomposition, but they are designed for seasonal data. Since we only have one year of daily data, we cannot reliably estimate a seasonal component. These functions are available in the R package `stats`, they need an object of the class `ts` (time series). The following code shows how to use the `stl` function to decompose the log PM2.5 series into trend, seasonal and remainder components. However, for this precise example is not recommended to use it since we only have one year of data.

In 'decompose()', type chooses how the components combine: - Additive: series = trend + seasonal + remainder. Use it when seasonal swings are roughly the same size over time. - Multiplicative: series = trend × seasonal × remainder. Use it when seasonal swings grow or shrink with the series level; the data must be positive. In 'stl()', s.window = "periodic" makes the seasonal pattern the same for every cycle. With frequency = 7, it estimates one weekly pattern and repeats it unchanged. This is different from STL's numeric s.window, which controls how flexibly the seasonal pattern can change over time.

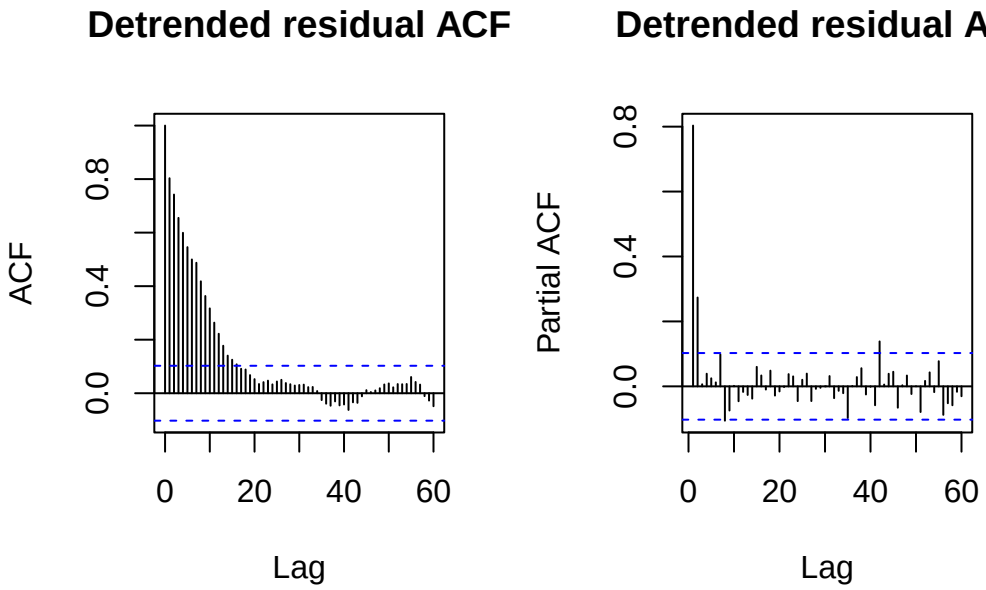
```
pm_ts <- ts(d$log_pm25, frequency = 7)

parts <- decompose(pm_ts, type = "additive")
plot(parts)

part <- stl(pm_ts, s.window = "periodic", robust = TRUE)
plot(part)
```

Correlogram

```
par(mfrow=c(1,2))
e <- residuals(lm(y~t))
acf(e,lag.max=60,main="Detrended residual ACF")
pacf(e,lag.max=60,main="Detrended residual ACF")
```

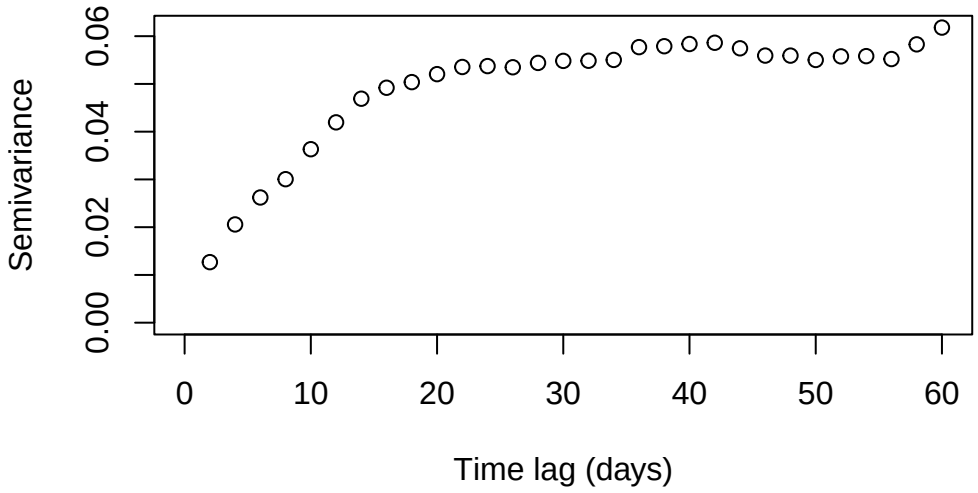


The ACF (left) starts very high and declines gradually over many lags. Even after removing a linear trend, nearby residuals remain strongly correlated, so the series still has substantial temporal dependence. The PACF (right) has a dominant spike at lag 1 and a smaller one at lag 2, while most later spikes stay within the blue confidence limits.

Variogram

```
time_coords = cbind(day = d$day, constant = 1)
v <- geoR::variog(
  coords = time_coords,
  data = d$log_pm25,
  uvec = seq(0, 60, by = 2),
  trend = "cte",
  messages = FALSE
)

plot(v, xlab = "Time lag (days)", ylab = "Semivariance")
```



ML fits for five models

Autoregressive models

```
d = as.data.frame(d)
fit1<- spaMM::fitme(
  log_pm25 ~ day + AR1(1 | day),
  data = d,
  family = gaussian(),
  method = "ML"
)
```

Gaussian process models with Matérn covariance

```
fit2 <- spaMM::fitme(
  log_pm25 ~ day + Matern(1 | day),
  data = d,
  family = gaussian(),
  fixed = list(nu = 0.5),
  method = "ML"
)

fit3 <- spaMM::fitme(
```

```
  log_pm25 ~ day + Matern(1 | day),
  data = d,
  family = gaussian(),
  fixed = list(nu = 1.5),
  method = "ML"
)

fit4 <- spaMM::fitme(
  log_pm25 ~ day + Matern(1 | day),
  data = d,
  family = gaussian(),
  fixed = list(nu = 2.5),
  method = "ML"
)
```

Comparison of the models

```
aic1 = AIC(fit1)[1]
aic2 = AIC(fit2)[1]
aic3 = AIC(fit3)[1]
aic4 = AIC(fit4)[1]

c(aic1, aic2, aic3, aic4)
```

```
       marginal AIC:        marginal AIC:        marginal AIC:
           -412.9479            -412.9479            -407.9355
       marginal AIC:
           -403.3171
```

```
rmse <- function(observed, predicted) {
  sqrt(mean((observed - predicted)^2, na.rm = TRUE))
}

rmse1 <- rmse(d$log_pm25, fitted(fit1))
rmse2 <- rmse(d$log_pm25, fitted(fit2))
rmse3 <- rmse(d$log_pm25, fitted(fit3))
rmse4 <- rmse(d$log_pm25, fitted(fit4))
```

```r
c(rmse1, rmse2, rmse3, rmse4)
```

```
[1] 0.05677732 0.05677739 0.08377598 0.08630789
```

```r
results <- data.frame(
  model = c("AR1",
            "Matern nu=0.5", "Matern nu=1.5", "Matern nu=2.5"),
  AIC = c(aic1, aic2, aic3, aic4),
  RMSE = c(rmse1, rmse2, rmse3, rmse4)
)

results
```

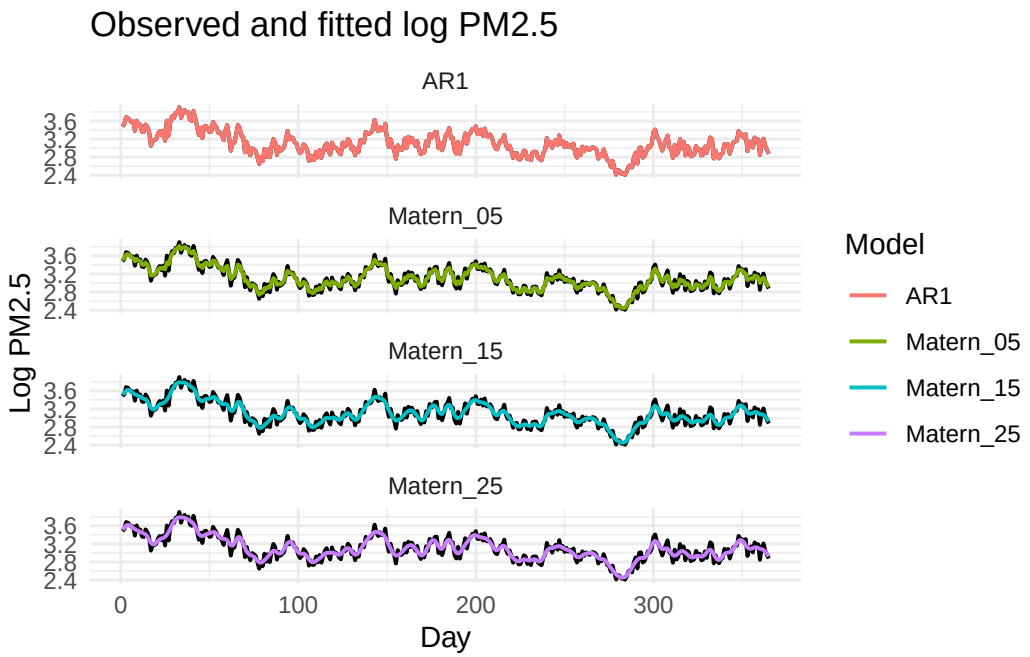
```
          model       AIC       RMSE
1           AR1 -412.9479 0.05677732
2 Matern nu=0.5 -412.9479 0.05677739
3 Matern nu=1.5 -407.9355 0.08377598
4 Matern nu=2.5 -403.3171 0.08630789
```

```r
plot_data <- data.frame(
  day = d$day,
  observed = d$log_pm25,
  AR1 = as.numeric(fitted(fit1) + residuals(fit1, type = "response")),
  Matern_05 = as.numeric(fitted(fit2)),
  Matern_15 = as.numeric(fitted(fit3)),
  Matern_25 = as.numeric(fitted(fit4))
)

plot_long <- tidyr::pivot_longer(
  plot_data,
  cols = -c(day, observed),
  names_to = "model",
  values_to = "predicted"
)

ggplot(plot_long, aes(x = day)) +
  geom_line(aes(y = observed), color = "black", linewidth = 0.7) +
  geom_line(aes(y = predicted, color = model), linewidth = 0.7) +
  facet_wrap(~model, ncol = 1) +
  labs(
```

```
    x = "Day",
    y = "Log PM2.5",
    title = "Observed and fitted log PM2.5",
    color = "Model"
  ) +
  theme_minimal()
```



Hold-out validation

We reserve the final 20% of the daily PM2.5 observations as a hold-out test set. The models are fitted using only the first 80% of the time series, then used to predict the held-out period. We compare their predictive accuracy using test-set RMSE on the log PM2.5 scale; lower RMSE indicates predictions closer to the observed values.

```
# Hold out the last 20% of observations
n_train <- floor(0.8 * nrow(d))
train <- d[1:n_train, ]
test  <- d[(n_train + 1):nrow(d), ]

# Fit each model using training data only
fit1_train <- spaMM::fitme(
  log_pm25 ~ day + AR1(1 | day),
```

```r
  data = train,
  family = gaussian(),
  method = "ML"
)

fit2_train <- spaMM::fitme(
  log_pm25 ~ day + Matern(1 | day),
  data = train,
  family = gaussian(),
  fixed = list(nu = 0.5),
  method = "ML"
)

fit3_train <- spaMM::fitme(
  log_pm25 ~ day + Matern(1 | day),
  data = train,
  family = gaussian(),
  fixed = list(nu = 1.5),
  method = "ML"
)

fit4_train <- spaMM::fitme(
  log_pm25 ~ day + Matern(1 | day),
  data = train,
  family = gaussian(),
  fixed = list(nu = 2.5),
  method = "ML"
)

# Predict the held-out observations
predictions <- data.frame(
  day = test$day,
  observed = test$log_pm25,
  AR1 = predict(fit1_train, newdata = test),
  Matern_05 = predict(fit2_train, newdata = test),
  Matern_15 = predict(fit3_train, newdata = test),
  Matern_25 = predict(fit4_train, newdata = test)
)

# Test-set RMSE
rmse <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2, na.rm = TRUE))
```

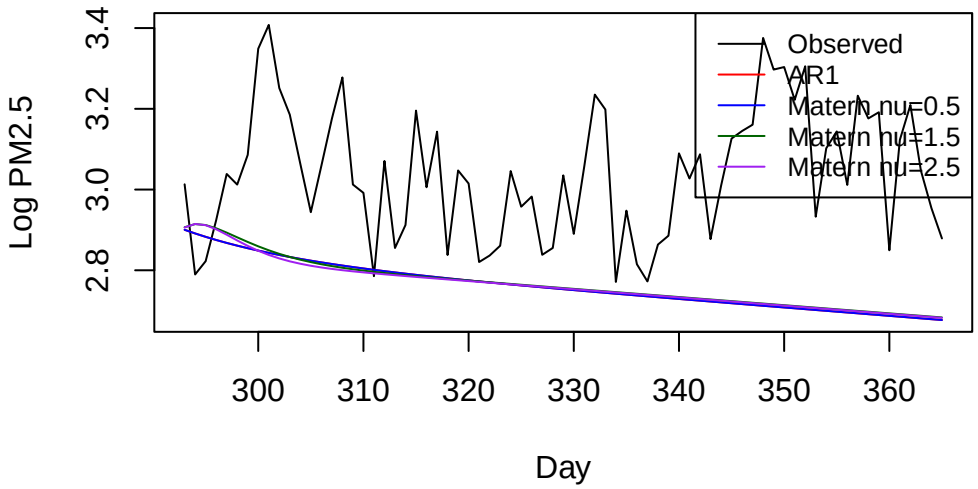
```
}

data.frame(
  model = c("AR1", "Matern nu=0.5", "Matern nu=1.5", "Matern nu=2.5"),
  RMSE = c(
    rmse(predictions$observed, predictions$AR1),
    rmse(predictions$observed, predictions$Matern_05),
    rmse(predictions$observed, predictions$Matern_15),
    rmse(predictions$observed, predictions$Matern_25)
  )
)
```

```
          model      RMSE
1           AR1 0.3282199
2 Matern nu=0.5 0.3282200
3 Matern nu=1.5 0.3251470
4 Matern nu=2.5 0.3274219
```

```
# Plot hold-out observations and predictions
matplot(
  predictions$day,
  predictions[, c("observed", "AR1", "Matern_05", "Matern_15", "Matern_25")],
  type = "l",
  lty = 1,
  col = c("black", "red", "blue", "darkgreen", "purple"),
  xlab = "Day",
  ylab = "Log PM2.5"
)

legend(
  "topright",
  legend = c("Observed", "AR1", "Matern nu=0.5", "Matern nu=1.5", "Matern nu=2.5"),
  col = c("black", "red", "blue", "darkgreen", "purple"),
  lty = 1,
  cex = 0.8
)
```


Log PM2.5
2.8
3.0
3.2
3.4
300
310
320
330
340
350
360
Day
Observed
AR1
Matern nu=0.5
Matern nu=1.5
Matern nu=2.5